

Retail Shelf KPI Estimation with YOLOv8 and ONNX Runtime RPC Dataset Prototype (15 Classes)

Khaled Nabil

February 8, 2026

Abstract

This project builds an end-to-end pipeline for retail shelf analytics using object detection. Starting from the Retail Product Checkout (RPC) dataset, we train a lightweight YOLOv8 model on a filtered subset of 15 high-frequency product classes, export the detector to ONNX, and run batch inference to generate business-facing shelf KPIs such as market share by facings, dominance, and out-of-stock (OOS) rates. The solution is designed to run reliably in cloud notebooks (Kaggle/Google Colab) and supports an inference-only path for evaluation.

1 Business Problem and KPI Definitions

Retail teams often need to quantify shelf presence (“facings”) and product availability from photos. Given an image of a shelf, our goal is to detect products, count facings by brand, and compute KPIs that map directly to common category-management questions:

- **Facings:** number of detected product instances per brand (proxy for shelf space).
- **Market share by facings:** $\text{share} = \frac{\text{our facings}}{\text{our facings} + \text{competitor facings}}$.
- **OOS rate:** fraction of images where the “OurBrand” product is not detected.
- **Dominance ratio:** $\frac{\text{our facings}}{\text{competitor facings}}$, a compact indicator of whether we are leading on-shelf.

This framing allows model outputs to be interpreted as operational metrics (availability and relative presence), not only as detection scores.

2 Dataset and Problem Scope

We use the **Retail Product Checkout (RPC)** dataset (COCO-style annotations). To keep training and iteration fast while maintaining meaningful evaluation, we restrict the task to the **top 15 most frequent classes** (filtered from 200 total classes) and sample **30%** of the filtered data for training. We also reserve a **50-image holdout set** for final KPI computation.

For brand analytics, we define a simple mapping from product classes to brand labels using a trial configuration:

- **OurBrand** class: `177_tissue`
- **Competitor** classes: 14 remaining classes in the filtered set

3 Modeling Approach

3.1 Why YOLOv8n

We selected **YOLOv8n** as the baseline detector for three reasons:

1. **Speed**: the “nano” variant is suited to notebook-based training and CPU-friendly deployment.
2. **Strong accuracy/effort tradeoff**: fast convergence for a prototype, especially after class filtering.
3. **Exportability**: easy ONNX export for portable inference.

3.2 Training Configuration

The notebook uses a consistent configuration across Kaggle and Colab:

- Epochs: 30 with early stopping patience = 10
- Batch size: 16
- Image size: 416×416
- Classes: 15 (filtered); holdout size: 50 images

These values were chosen to reduce runtime while preserving quality for a 15-class detection task.

4 Deployment-Oriented Inference (ONNX Runtime)

To support evaluation and CPU execution, the trained model is exported to **ONNX** and inference is performed using **ONNX Runtime**. Post-processing uses confidence filtering and Non-Maximum Suppression (NMS) with:

- Confidence threshold: 0.25
- IoU threshold: 0.45

This inference path is intended to be deterministic and portable across environments, and it cleanly separates training from the evaluation flow.

5 Results

5.1 Detection Quality (Training/Validation)

Final-epoch validation metrics from the training log (epoch 30) indicate strong detection performance on the filtered task:

- Precision: 0.9808
- Recall: 0.9804
- mAP@0.50: 0.9896
- mAP@0.50:0.95: 0.8287

These values suggest the model is accurate enough to support downstream KPI aggregation without excessive noise.

5.2 Business KPIs on Holdout Images

We compute KPIs on the 50 holdout images produced by the pipeline. The aggregate results are:

Metric	Value
Total images	50
Total detections (all classes)	117
Average detections per shelf	2.34
OurBrand facings	13
Competitor facings	104
OurBrand market share (facings)	11.11%
OOS images count (OurBrand absent)	44
OOS rate	88.0%
Dominance ratio (OurBrand / Competitor)	0.125

Table 1: Aggregate shelf KPIs computed on the 50-image holdout set.

Interpretation. The KPI profile indicates that, under this trial brand mapping, **OurBrand is frequently absent** on the shelf images (OOS rate 88%) and is **dominated by competitors** in terms of detected facings (dominance ratio 0.125). Even with high detection metrics, these business KPIs can highlight assortment or execution issues.

6 Engineering Decisions and Challenges

6.1 Cross-Platform Notebook Reliability

A key requirement was to run consistently in Kaggle and Google Colab. The notebook standardizes a workspace root (e.g., `/kaggle/working` vs. `/content`) and uses the same relative project structure: `config/`, `data/` (holdout), `runs/`, and `output/`. This reduces path-related failures and makes outputs predictable for evaluators.

6.2 Challenges and Mitigations

During development we encountered a set of practical issues that typically block reproducibility in notebook-based ML projects. Below we summarize what went wrong and the concrete mitigation implemented in the pipeline.

- **Kaggle vs. Colab path mismatches.** The original workflow relied on Kaggle-specific paths (e.g., `/kaggle/input/...`) and assumptions about where outputs are written. *Mitigation:* add platform detection and route all reads/writes through a single workspace root (`/kaggle/working` or `/content`) with a fixed project folder layout.
- **Dataset availability differences (training vs. evaluation).** In Colab, the full RPC dataset may not be present (judges often run inference only). *Mitigation:* the setup stage checks for required COCO JSON files and clearly gates training cells; if missing, the notebook still supports inference provided the model weights and holdout images exist locally.

- **Inference-only evaluation constraints.** Evaluation should not require re-training (time/compute) and must run deterministically. *Mitigation:* provide an explicit inference-only path that consumes `best.onnx`, `class_mapping.json`, and the holdout images, producing the same KPI artifacts as the full pipeline.
- **ONNX post-processing consistency.** Exporting to ONNX changes the runtime stack and can introduce inconsistencies if preprocessing/NMS differs from training-time assumptions. *Mitigation:* standardize inference preprocessing (resize/letterbox to 416×416) and apply a fixed post-processing policy (`conf=0.25`, `IoU=0.45`) before KPI aggregation.
- **Label mapping and class filtering correctness.** Filtering from 200 RPC classes to 15 requires a deterministic `RPC → YOLO` index mapping; errors here silently corrupt metrics. *Mitigation:* generate and persist mappings (`filtered_classes.yaml`, `class_mapping.json`) once, then reuse the same mapping for training, inference, and KPI computation.
- **Resource limits and runtime predictability.** Full-scale training is expensive in notebook environments and may exceed session limits. *Mitigation:* constrain scope (15 classes, 30% sample) and use a small model (YOLOv8n) with early stopping to reach strong accuracy quickly.

6.3 Inference-Only “Judges” Path

To support fast evaluation, the pipeline includes an inference-only mode that requires only:

- `best.onnx` (and optionally `best.pt`) under `runs/train/weights/`
- `class_mapping.json` under `config/`
- holdout images under `data/`

This design decouples training time from evaluation while still producing the same KPI artifacts.

6.4 Why Class Filtering

RPC has 200 classes; training a full detector is expensive and unnecessary for demonstrating the KPI pipeline. Filtering to 15 high-frequency classes reduces class imbalance, speeds up training, and provides a stable basis for validating the end-to-end KPI computation.

7 Limitations and Future Work

- **Brand mapping is simplified:** a single class represents “OurBrand”. A production system would group multiple SKUs per brand and handle brand families.
- **OOS detection is detection-dependent:** missed detections can inflate OOS. Future work could add temporal smoothing, shelf-layout priors, or threshold calibration.
- **Facings vs. shelf share:** facings are a proxy. A next step is estimating planogram share (e.g., by bounding-box area or shelf segmentation).
- **Domain shift:** store lighting, occlusion, and camera angle affect performance. Further augmentation and targeted finetuning would improve robustness.

8 Reproducibility Notes

The project produces machine-readable outputs for verification, including `output/aggregate_kpis.json`, per-image KPI CSVs, and prediction logs. Training curves and metrics are tracked in `runs/train/results.csv`. The same notebook supports both full training and inference-only execution.